

OmniLitter

Sistema multiplataforma para la gestión y análisis de datos de
contaminación por residuos



Escuela Técnica Superior de
Ingeniería Informática

Trabajo Fin de Grado

Grado: Ingeniería Informática - Ingeniería del Software

Centro: ETSII — Universidad de Sevilla

Curso académico: 2025/26

Autores:

- Alejandro Castilla — alecasbar@alum.us.es
- Ignacio Martínez Díaz — ignmardial@alum.us.es

Tutor académico: Cruz Mata, Fermín

Cotutor: Ortega Rodríguez, Francisco Javier

Cliente / entidad colaboradora: Daniel González Fernández /
Universidad de Cádiz

Índice

1. Introducción	5
2. Descripción del Proyecto	6
2.1. Objetivos	6
3. Arquitectura del Sistema	7
3.1. Visión general	7
4. Modelo de Datos en CouchDB	8
4.1. Base de datos	8
4.2. Documento de película	8
5. Funcionalidades de la Aplicación	9
5.1. Búsqueda de películas	9
6. API REST del Backend	10
7. Diseño de la Solución	11
7.1. Visión General de la Arquitectura	11
7.1.1. Justificación de las decisiones arquitectónicas principales	12
7.2. Diseño de la Base de Datos	12
7.2.1. Modelo relacional	12
7.2.2. Descripción de entidades	13
7.2.3. Decisiones de diseño de la base de datos	15
7.3. Diseño de la API REST	16
7.3.1. Autenticación y autorización	17
7.4. Diseño del Portal Web	18
7.4.1. Arquitectura de componentes	18
7.4.2. Gestión de estado	18
7.4.3. Flujo de navegación y control de acceso por rol	18
7.5. Diseño de la Aplicación Móvil	19
7.5.1. Stack tecnológico y estructura de pantallas	19
7.5.2. Gestión de estado global	19
7.5.3. Base de datos local con WatermelonDB	19
7.5.4. Flujo de captura de una observación	19
7.6. Mecanismo de Sincronización Offline	20
7.6.1. Patrón Outbox Queue	20
7.6.2. Estados de sincronización	20
7.6.3. Resolución de conflictos	21
7.6.4. Garantía de identificadores únicos en entorno offline	21
A. Declaración del Uso de IA	23

A.1. Prompts utilizados	23
A.2. Autoría y revisión crítica	25

Índice de cuadros

1.	Descripción de entidades del modelo de datos.	13
2.	Principales <i>endpoints</i> de la API REST de OmniLitter.	16
3.	Estados de sincronización y su significado.	21
4.	Detalle del uso de IA por parte del trabajo.	23

Índice de figuras

1. Arquitectura general propuesta para OmniLitter. 11
2. Diagrama entidad-relación del sistema OmniLitter. 13

1. Introducción

Llevar la cuenta de todo lo que uno ve se ha convertido en algo

2. Descripción del Proyecto

2.1. Objetivos

3. Arquitectura del Sistema

3.1. Visión general

4. Modelo de Datos en CouchDB

4.1. Base de datos

4.2. Documento de película

5. Funcionalidades de la Aplicación

5.1. Búsqueda de películas

6. API REST del Backend

7. Diseño de la Solución

Este capítulo describe las decisiones de diseño adoptadas para materializar los requisitos funcionales y no funcionales de OmniLitter en una arquitectura concreta. Se abordan, en orden de abstracción descendente, la arquitectura general del sistema, el diseño de la base de datos, el contrato de la API REST, la estructura de componentes del portal web y de la aplicación móvil, y el mecanismo de sincronización *offline*.

7.1. Visión General de la Arquitectura

OmniLitter sigue una arquitectura de **tres capas** clásica adaptada a un contexto multi-plataforma con requisito de operación sin conexión:

1. **Capa de presentación.** Comprende dos clientes independientes: el portal web, orientado a la consulta y gestión de datos por parte de investigadores y administradores, y la aplicación móvil, orientada a la captura de observaciones en campo.
2. **Capa de lógica de negocio.** Una API REST centralizada implementada en Node.js + Express actúa como único punto de acceso a los datos, aplica las reglas de negocio y gestiona la autenticación mediante *tokens* JWT.
3. **Capa de persistencia.** PostgreSQL almacena el estado canónico del sistema. Adicionalmente, la aplicación móvil mantiene una base de datos local con WatermelonDB (capa sobre SQLite) que permite operar sin conexión y sincronizarse en diferido.

La figura 1 muestra un diagrama de la arquitectura del sistema.

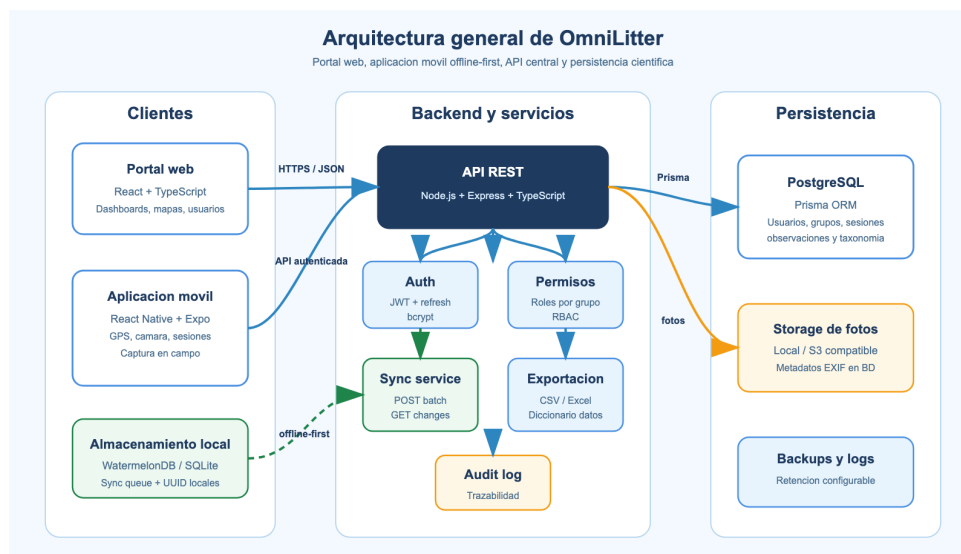


Figura 1: Arquitectura general propuesta para OmniLitter.

7.1.1. Justificación de las decisiones arquitectónicas principales

API REST como único punto de acceso. Centralizar la lógica de negocio en la API evita duplicidades entre los dos clientes y facilita la incorporación de nuevos consumidores en el futuro (p.ej., exportaciones automatizadas por *scripts* externos o integraciones con repositorios científicos).

Separación entre portal web y aplicación móvil. Aunque comparten la misma API, los dos clientes tienen requisitos de experiencia de usuario muy distintos: el portal prioriza la visualización analítica y la gestión, mientras que la app móvil prioriza la captura rápida en condiciones de campo adversas (luz solar, guantes, sin conexión). Mantenerlos como proyectos independientes permite evolucionar cada uno a su propio ritmo.

Base de datos local en el móvil. El trabajo de campo se realiza habitualmente en zonas costeras, ribereñas o forestales con cobertura de red intermitente o nula. La decisión de embeber una base de datos local en el dispositivo es, por tanto, un requisito no negociable para la viabilidad del producto.

7.2. Diseño de la Base de Datos

7.2.1. Modelo relacional

El modelo de datos de OmniLitter se compone de once entidades que se agrupan en tres áreas funcionales:

- **Gestión de acceso:** USER, GROUP, MEMBERSHIP.
- **Organización científica:** CAMPAIGN, SURVEY_SESSION.
- **Datos de campo:** OBSERVATION, PHOTO, TAXONOMY_ITEM, FAVORITE, AUDIT_LOG.

La figura 2 muestra el diagrama entidad-relación del sistema.

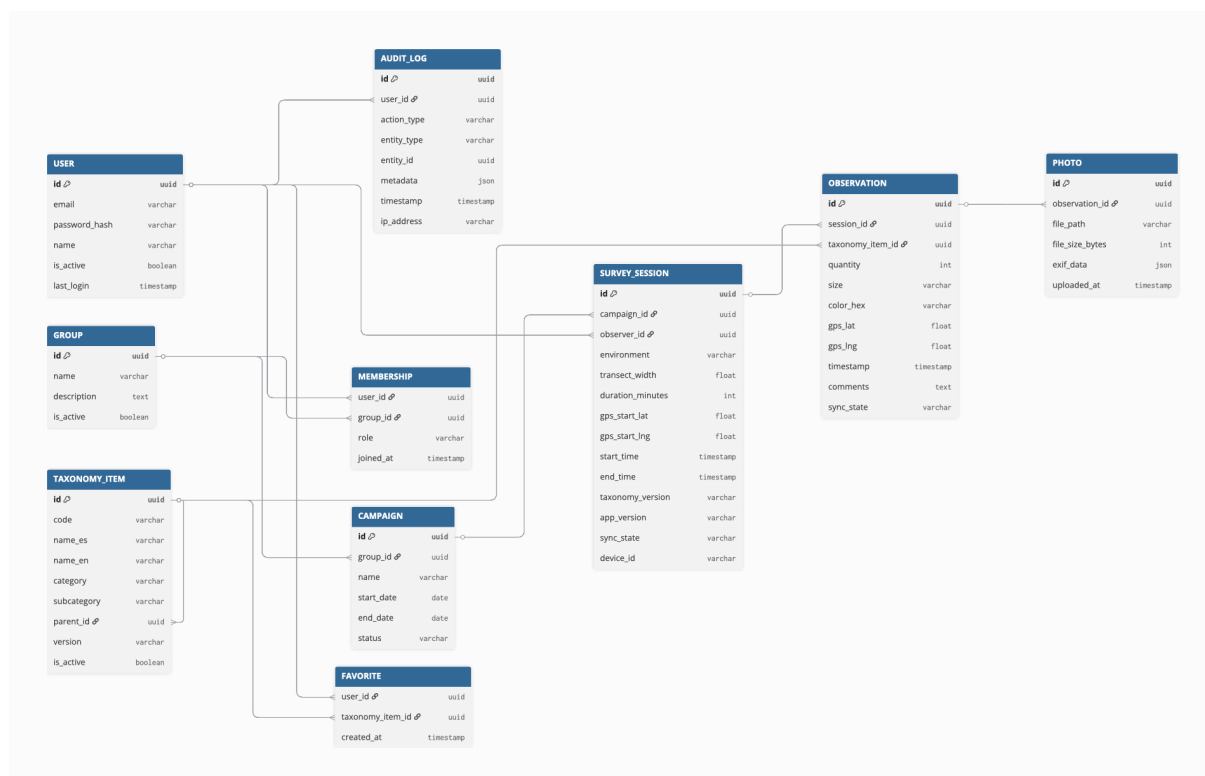


Figura 2: Diagrama entidad-relación del sistema OmniLitter.

7.2.2. Descripción de entidades

La tabla 1 describe los atributos clave de cada entidad y su propósito en el sistema.

Cuadro 1: Descripción de entidades del modelo de datos.

Entidad	Atributos principales	Descripción
USER	id (UUID), email, password_hash, name, is_active, last_login	Representa a cada persona registrada en el sistema. Las contraseñas se almacenan como <i>hash</i> bcrypt y nunca en texto plano.
GROUP	id (UUID), name, description, is_active	Equipo de investigación. Un grupo puede ser archivado sin eliminar sus datos históricos, preservando la trazabilidad científica.
MEMBERSHIP	user_id (FK), group_id (FK), role (admin editor viewer), joined_at	Tabla de unión que asocia usuarios con grupos asignándoles un rol. Un mismo usuario puede tener roles distintos en grupos distintos.

Entidad	Atributos principales	Descripción
CAMPAIGN	id (UUID), group_id (FK), name, start_date, end_date, status	Proyecto científico de muestreo. Agrupa varias sesiones de campo bajo un objetivo común (p.ej., «Monitoreo anual Playa de la Caleta 2026»).
SURVEY_SESSION	id (UUID), campaign_id (FK), observer_id (FK), environment, transect_width, duration_minutes, gps_start_lat/lng, start_time, end_time, taxonomy_version, app_version, sync_state, device_id	Sesión de muestreo individual en campo. Registra todos los metadatos científicos necesarios para la reproducibilidad: protocolo, versión de taxonomía utilizada y versión de la aplicación. El campo sync_state gestiona el ciclo de vida de la sincronización.
OBSERVATION	id (UUID), session_id (FK), taxonomy_item_id (FK), quantity, size, color_hex, gps_lat/lng, timestamp, comments, sync_state	Registro de un ítem de residuo observado. Constituye la unidad mínima de dato científico del sistema.
PHOTO	id (UUID), observation_id (FK), file_path, file_size_bytes, exif_data (JSONB), uploaded_at	Imagen asociada a una observación. Los metadatos EXIF (coordenadas GPS embebidas en la imagen, fecha de captura, modelo de cámara) se almacenan en formato JSONB para máxima flexibilidad.
TAXONOMY_ITEM	id (UUID), code, name_es, name_en, category, subcategory, parent_id (FK self-ref), version, is_active	Catálogo de tipos de residuo basado en la <i>Joint List of Litter Categories</i> . La estructura jerárquica auto-referenciada (categoría → subcategoría → ítem) permite navegar la taxonomía sin conexión, ya que se descarga completa al iniciar sesión.

Entidad	Atributos principales	Descripción
FAVORITE	<code>user_id</code> (FK), <code>taxonomy_item_id</code> (FK), <code>created_at</code>	Ítems de taxonomía marcados como favoritos por el usuario para agilizar la captura de observaciones en campo.
AUDIT_LOG	<code>id</code> , <code>user_id</code> (FK), <code>action_type</code> , <code>entity_type</code> , <code>entity_id</code> , <code>metadata</code> (JSONB), <code>timestamp</code> , <code>ip_address</code>	Registro de auditoría que almacena las acciones relevantes sobre el sistema (inicio de sesión, creación y edición de observaciones, exportaciones de datos, cambios de configuración). Garantiza la trazabilidad exigida en entornos de investigación científica.

7.2.3. Decisiones de diseño de la base de datos

UUID como clave primaria universal. Todos los identificadores primarios son UUID v4 generados en el cliente (ya sea el navegador o el dispositivo móvil). Esto resuelve el problema de colisiones de claves en escenarios de creación offline: dos dispositivos diferentes que creen una observación sin conexión obtendrán identificadores globalmente únicos que no entrarán en conflicto cuando se sincronicen con el servidor.

Rol en MEMBERSHIP, no en USER. El rol de un investigador no es una propiedad intrínseca de la persona, sino de su relación con un grupo concreto. Un mismo investigador puede ser administrador en su grupo principal y colaborador invitado en otro grupo. Modelar el rol en la tabla de unión es, por tanto, la decisión correcta desde el punto de vista semántico.

Versión de taxonomía en cada sesión y observación. La taxonomía de residuos evoluciona con el tiempo a medida que la comunidad científica añade o reorganiza categorías. Registrar la versión exacta utilizada en cada sesión permite reconstruir con exactitud qué definición se aplicó a cada observación, lo que es un requisito básico de reproducibilidad científica.

JSONB para metadatos heterogéneos. Los campos `exif_data` y `metadata` utilizan el tipo JSONB de PostgreSQL para almacenar datos cuya estructura puede variar (distintos modelos de cámara incluyen campos EXIF diferentes, distintas acciones de auditoría tienen distintos contextos). Esta decisión evita la proliferación de columnas opcionales con valor nulo y preserva la flexibilidad para añadir nuevos atributos sin migrar el esquema.

7.3. Diseño de la API REST

La API sigue los principios REST: interfaz uniforme, comunicación sin estado, recursos identificados por URI y representaciones en formato JSON. Todos los *endpoints* protegidos requieren el encabezado `Authorization: Bearer <token>`, donde el *token* es un JWT firmado emitido en el *endpoint* de autenticación.

La tabla 2 recoge los *endpoints* principales agrupados por recurso.

Cuadro 2: Principales *endpoints* de la API REST de OmniLitter.

Método	Ruta	Protegido	Descripción
<i>Autenticación</i>			
POST	/api/auth/login	No	Emite JWT de acceso y refresco
POST	/api/auth/refresh	No	Renueva el JWT de acceso
POST	/api/auth/logout	Sí	Invalida el <i>token</i> de refresco
<i>Usuarios</i>			
GET	/api/users	Sí (admin)	Lista todos los usuarios
POST	/api/users	Sí (admin)	Crea un nuevo usuario
GET	/api/users/:id	Sí	Obtiene un usuario
PATCH	/api/users/:id	Sí (admin)	Actualiza datos o estado
<i>Grupos</i>			
GET	/api/groups	Sí	Lista grupos del usuario autenticado
POST	/api/groups	Sí (admin)	Crea un grupo
PATCH	/api/groups/:id	Sí (admin)	Edita nombre/descripción
DELETE	/api/groups/:id/archive	Sí (admin)	Archiva un grupo
POST	/api/groups/:id/members	Sí (admin)	Añade miembro
DELETE	/api/groups/:id/members/:uid	Sí (admin)	Elimina miembro
<i>Sesiones de muestreo</i>			
GET	/api/sessions	Sí	Lista sesiones accesibles
POST	/api/sessions	Sí	Crea una sesión
GET	/api/sessions/:id	Sí	Obtiene una sesión con observaciones

Método	Ruta	Protegido	Descripción
PATCH	/api/sessions/:id	Sí	Actualiza metadatos
<i>Observaciones</i>			
GET	/api/sessions/:id/observations	Sí	Lista observaciones de sesión
POST	/api/sessions/:id/observations	Sí	Registra una observación
GET	/api/observations	Sí	Lista todas (filtros opcionales)
<i>Fotos</i>			
POST	/api/observations/:id/photos	Sí	Sube una foto (multipart)
GET	/api/photos/:id	Sí	Sirve el archivo de imagen
<i>Taxonomía</i>			
GET	/api/taxonomy	Sí	Descarga la taxonomía completa
GET	/api/taxonomy?version=:v	Sí	Descarga versión específica
<i>Sincronización</i>			
POST	/api/sync/batch	Sí	Sube cola de registros pendientes
GET	/api/sync/changes	Sí	Descarga cambios desde last_sync
<i>Exportación</i>			
GET	/api/export/csv	Sí (admin)	Exporta observaciones en CSV científico

7.3.1. Autenticación y autorización

El sistema implementa autenticación basada en JWT con dos *tokens*: un **token de acceso** de corta duración (15 minutos) y un **token de refresco** de larga duración (7 días). El token de acceso se envía en cada petición en el encabezado **Authorization**; el de refresco se utiliza exclusivamente para solicitar un nuevo token de acceso cuando el anterior expira.

La autorización sigue un modelo de control de acceso basado en roles (RBAC) a nivel de grupo. El *middleware* de autorización verifica, para cada petición, que el usuario autenticado posee el rol requerido en el grupo al que pertenece el recurso solicitado. Esto permite que una misma persona sea administradora en un grupo y colaboradora en otro,

con permisos diferentes en cada contexto.

7.4. Diseño del Portal Web

7.4.1. Arquitectura de componentes

El portal web está implementado como una *Single Page Application* (SPA) con React 18 y TypeScript, empaquetada con Vite y estilizada con Tailwind CSS. La navegación entre páginas es gestionada por React Router, que define dos niveles de rutas:

- **Rutas públicas:** la página de inicio (/) y el formulario de inicio de sesión (/login), accesibles sin autenticación.
- **Rutas protegidas:** todas las páginas bajo /app/*, protegidas por un componente `PrivateRoute` que verifica el estado de autenticación y redirige al *login* si no existe sesión activa.

La estructura de componentes sigue una jerarquía de tres niveles: el `Layout` general (barra lateral, encabezado, selector de grupo activo) envuelve las páginas individuales, que a su vez utilizan componentes reutilizables (tarjetas de estadística, tablas filtrables, modales de confirmación).

7.4.2. Gestión de estado

El portal web utiliza `localStorage` como mecanismo de persistencia de la sesión del usuario entre recargas de página, almacenando el JWT de acceso, el nombre del usuario, su rol y el grupo activo seleccionado. Este enfoque es suficiente para una SPA sin estado de servidor, y se complementará con manejo de *refresh token* en la implementación real.

La comunicación entre el selector de grupo activo (ubicado en la barra lateral) y el componente Dashboard se implementa mediante un evento personalizado del DOM (`omnilitter:groupChange`) lo que evita prop-drilling innecesario sin requerir una solución de gestión de estado global.

7.4.3. Flujo de navegación y control de acceso por rol

Las secciones de **Grupos** y **Usuarios** son exclusivas del rol de administrador y no aparecen en la barra de navegación para los colaboradores. El Dashboard ajusta el contenido mostrado según el rol: los administradores ven estadísticas globales del grupo activo, mientras que los colaboradores ven únicamente sus propias métricas. La tabla de observaciones filtra los registros por autor para los colaboradores y muestra todos para los administradores.

7.5. Diseño de la Aplicación Móvil

7.5.1. Stack tecnológico y estructura de pantallas

La aplicación móvil está construida con **React Native** y **Expo bare workflow**. La elección del *bare workflow* frente al *managed workflow* responde a la necesidad de acceso completo a las APIs nativas del dispositivo: GPS continuo con expo-location, captura de imagen con metadatos EXIF con expo-camera, y sincronización en segundo plano con expo-background-fetch.

La navegación está organizada en dos niveles:

- **Navegación de primer nivel (tab bar):** cinco pestañas inferiores (Inicio, Sesiones, Mapa, Sincronización, Perfil), visibles únicamente en las pantallas principales.
- **Navegación de pila (stack):** flujos de varios pasos como la creación de una nueva sesión o la captura de una observación, que se presentan sin el *tab bar*.

7.5.2. Gestión de estado global

El estado global de la aplicación se gestiona mediante el patrón **Context + useReducer** de React, sin dependencias externas de gestión de estado. El contexto **AppContext** expone el estado completo (sesiones activas, tipos de residuo, datos del usuario, estado de sincronización) y un *dispatcher* con acciones tipadas en TypeScript. Este patrón proporciona la predictibilidad de Redux con una superficie de código significativamente menor, adecuada para la escala del proyecto.

7.5.3. Base de datos local con WatermelonDB

WatermelonDB actúa como base de datos local sobre SQLite. Se elige frente a SQLite directo por tres razones: (1) está diseñado explícitamente para el patrón *offline-first* con sincronización en diferido; (2) ofrece un sistema de consultas reactivo basado en observables que actualiza la UI automáticamente cuando cambian los datos; (3) su rendimiento en colecciones grandes (miles de observaciones) es significativamente superior al de SQLite gestionado manualmente.

El esquema local replica el subconjunto del modelo de datos necesario para la operación en campo: `survey_sessions`, `observations`, `photos` y `sync_queue`, además de la taxonomía completa cargada al iniciar sesión.

7.5.4. Flujo de captura de una observación

El flujo principal de la aplicación sigue esta secuencia:

1. El investigador inicia una nueva sesión de muestreo, especificando el nombre, el tipo de entorno y los parámetros del transecto.
2. Para cada ítem encontrado, abre la pantalla de nueva observación, selecciona el tipo de residuo a través del modal de taxonomía (organizado en cuadrícula visual por categorías e iconos), e introduce cantidad, tamaño, color y comentarios.
3. Opcionalmente captura una foto geolocalizada del residuo.
4. Al finalizar la sesión de campo, revisa el resumen y puede iniciar la sincronización manualmente o dejar que se realice en segundo plano al recuperar cobertura de red.

7.6. Mecanismo de Sincronización Offline

7.6.1. Patrón Outbox Queue

El mecanismo de sincronización implementa el patrón **Outbox Queue** (cola de salida), ampliamente utilizado en sistemas distribuidos con conectividad intermitente. El funcionamiento es el siguiente:

1. Cuando el usuario crea o modifica un registro (sesión u observación) en el dispositivo sin conexión, el registro se persiste en WatermelonDB con `sync_state = PENDING` y se inserta una entrada en la tabla `sync_queue` con el *payload* completo del cambio.
2. Cuando el dispositivo recupera conectividad, `expo-background-fetch` activa el proceso de sincronización, que lee la cola y realiza una petición `POST /api/sync/batch` con todos los registros pendientes.
3. El servidor procesa los registros en orden, los persiste en PostgreSQL y devuelve el estado canónico actualizado.
4. El cliente actualiza el `sync_state` de los registros procesados a `SYNCED` y descarga los cambios remotos realizados por otros usuarios mediante `GET /api/sync/changes?since={last_synced}`.
5. En caso de error en un registro concreto (conflicto de datos, registro inválido), ese registro pasa a `sync_state = ERROR` y se notifica al usuario, sin bloquear la sincronización del resto.

7.6.2. Estados de sincronización

Cada sesión y observación puede encontrarse en uno de cuatro estados, como muestra la tabla 3.

Cuadro 3: Estados de sincronización y su significado.

Estado	Significado
PENDING	El registro existe solo en el dispositivo; aún no se ha intentado sincronizar.
SYNCING	El registro está siendo procesado en la petición de sincronización actual.
SYNCED	El servidor ha confirmado la recepción y persistencia del registro.
ERROR	El servidor rechazó el registro (conflicto, datos inválidos). Requiere intervención manual o reintento.

7.6.3. Resolución de conflictos

La estrategia de resolución de conflictos adoptada es **last-write-wins** con granularidad de registro: si dos usuarios modifican la misma sesión de forma independiente mientras están sin conexión, prevalece la versión con el **timestamp** más reciente. Esta estrategia es suficiente para el modelo de uso de OmniLitter, donde cada sesión de muestreo pertenece a un único observador responsable y los conflictos genuinos son infrecuentes.

7.6.4. Garantía de identificadores únicos en entorno offline

Para que la estrategia Outbox Queue funcione sin servidor central asignando IDs, todos los identificadores del sistema son **UUID v4 generados en el cliente**. Esto garantiza que los registros creados simultáneamente en distintos dispositivos nunca colisionarán al sincronizarse, independientemente del orden de llegada al servidor.

Referencias

- [1] Apache Software Foundation. *Apache CouchDB Documentation*. 2024. Disponible en: <https://docs.couchdb.org> [Último acceso: abril 2026].

A. Declaración del Uso de IA

En cumplimiento con las buenas prácticas académicas y de transparencia, se declara que para la elaboración de este trabajo se ha contado con el apoyo de herramientas de Inteligencia Artificial Generativa, concretamente Claude y Claude Code, de Anthropic. El uso de dicha tecnología se ha limitado a las siguientes tareas:

- **Scaffolding inicial del proyecto.** Se ha utilizado Claude Code para generar la estructura base del proyecto, incluyendo la configuración del servidor Express, las rutas de la API REST y los componentes React iniciales. Este código fue revisado, ejecutado y modificado por los autores durante el desarrollo.
- **Diseño del modelo de datos.** Se ha empleado Claude como herramienta de consulta para contrastar decisiones de diseño sobre el esquema de documentos en CouchDB, la definición de las vistas MapReduce y la elección entre índices Mango y vistas para cada tipo de consulta.
- **Soporte técnico durante el desarrollo.** Se ha recurrido a la IA como apoyo en la resolución de problemas concretos surgidos durante la implementación, principalmente relacionados con la configuración de autenticación en CouchDB y la integración con la API de TMDb.
- **Revisión de texto.** Se ha realizado una revisión de la redacción de esta memoria con apoyo de IA para mejorar la fluidez y coherencia del texto. Este proceso se ha aplicado siempre sobre contenido escrito originalmente por los autores, manteniendo íntegras las decisiones técnicas, argumentaciones y conclusiones del trabajo.

El Cuadro 4 resume las partes del trabajo donde se utilizó IA y con qué propósito.

Cuadro 4: Detalle del uso de IA por parte del trabajo.

Parte del trabajo	Herramienta	Propósito
Elección del tema y stack	Claude (chat)	Evaluar opciones y decidir tecnologías.
Generación del código base	Claude Code	Scaffolding inicial de los 30 ficheros del proyecto.
Diseño del modelo CouchDB	Claude (chat)	Definir esquema de documentos y vistas MapReduce.
Depuración de configuración	Claude (chat)	Resolver problemas de autenticación con CouchDB.
Redacción de la memoria	Claude (chat)	Apoyo en la estructura y revisión del texto.

A.1. Prompts utilizados

A continuación se recogen los prompts empleados en las conversaciones con Claude, formulados para obtener orientación y apoyo sin delegar la toma de decisiones ni la imple-

mentación final en la herramienta.

Elección del tema y del stack

Estoy valorando usar CouchDB para un proyecto académico de catálogo personal de películas. Antes de decidirme, quiero entender qué ventajas e inconvenientes tiene frente a un enfoque relacional clásico en este contexto concreto, y qué stack web tendría sentido usar: qué tecnologías encajan bien con CouchDB, por qué, y qué sacrificaría con cada opción. No quiero que tomes la decisión por mí, sino que me des los argumentos para tomarla yo.

Scaffolding del proyecto

Necesito organizar la estructura de carpetas y ficheros de una aplicación con frontend y backend separados antes de empezar a implementar. Dime qué estructura tiene sentido, qué responsabilidad debería tener cada parte y cómo separarlas bien desde el principio. No hace falta que desarrolles el código, solo que me orientes sobre cómo plantearlo.

Modelo de datos en CouchDB

Estoy diseñando el modelo de datos de un catálogo de películas en CouchDB. Necesito decidir qué campos debería tener un documento de película y por qué tiene sentido representarlo así en un modelo documental. Ayúdame a razonar la estructura, señalando qué encaja bien en un documento JSON y qué habría que repensar, pero sin escribirme la solución entera.

Vistas MapReduce

Quiero implementar vistas MapReduce en CouchDB para agregar datos del catálogo: cuántas películas hay por género, cuántas por estado y cuál es la valoración media por categoría. Explícame cómo enfocar este tipo de consultas en CouchDB, qué hace la función map y qué papel juega el reduce en cada caso, pero déjame a mí escribir la implementación final.

Depuración de errores

Tengo un problema con CouchDB en mi proyecto: el servidor arranca pero no consigue conectar con la base de datos y devuelve un error de autenticación. Necesito entender qué puede estar fallando. Dime qué comprobaciones haría

para diagnosticar esto: dónde mirar, qué revisar y en qué orden, sin limitarte a darme una solución cerrada.

Redacción de la memoria

He redactado varios apartados de la memoria y quiero que suenen más claros y técnicamente correctos. Te paso el texto y necesito que me ayudes a reorganizarlo y reformularlo para que tenga un tono más académico y sea más fácil de leer. El contenido y las conclusiones son míos y no deben cambiar; solo quiero mejorar cómo están expresados.

A.2. Autoría y revisión crítica

Los autores declaran conocer y poder explicar en detalle todo el código y el contenido entregado. Las respuestas obtenidas de Claude se trataron en todo momento como material orientativo, revisado críticamente antes de incorporarse al proyecto. La selección del dominio, el diseño del modelo de datos, la implementación, la ejecución de pruebas, el análisis de resultados y la redacción definitiva de la memoria fueron responsabilidad de los autores.